Desarrollo de Aplicaciones Web

Expresiones Regulares

USC UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

Pablo García Tahoces

Expresiones Regulares

Durante el proceso de intercambio de datos con el servidor se han de intercambiar datos consistentes, para no corromper los datos almacenados en las bases de datos asociadas a aquel.

Solución → Previo al intercambio, los datos se analizan para comprobar que:

- Los campos obligatorios están cubiertos.
- El tipo de información solicitada es correcta.

Expresiones Regulares

Expresiones Regulares → permiten definir el patrón de una cadena de caracteres y utilizarlo como mecanismo de comprobación de la información.

Mecanismos de definición:

- Mediante barras inclinadas → */patrón/opciones.*

  Ej.- let miER= /[A-Z/i

- Mediante el constructor de la clase RegExp → *RegExp("patron", ["opciones"])*

  Ej.- let miER= new RegExp("[A-Z]", "i"])

Opciones:

- i.- búsqueda haciendo caso omiso de la distinción entre mayúsculas y minúsculas.
- g.- búsqueda global en toda la cadena.
- m.- permite búsqueda en textos multilínea.

Expresiones Regulares

Clase RegExp

Métodos:

- exec() → Testea coincidencias con una cadena de texto. Retorna la primera coincidencia.
- test() → Testea coincidencias con una cadena de texto. Retorna “true” o “false”
- toString() → Retorna el valor de la ER en formato de cadena. Heredado de *Object*.

Propiedades:

- global → Testea si el modificador “g” está activado.
- ignoreCase→ Testea si el modificador “i” está activado.
- lastIndex → Especifíca la posición en que comienza la siguiente coincidencia.
- multiline → Testea si el modificador “m” está activado.
- source → Retorna el valor de la ER en formato cadena.

Expresiones Regulares

Tipos de patrones:

- Cuantificadores
- Caracteres de escape
- Clases o conjuntos de caracteres
- Caracteres de marcado o posición
- Alternancias
- Agrupaciones
- Referencias inversas
- Patrones misceláneos

Expresiones Regulares

Patrones Validos:

- Cuantificadores.- permiten agregar datos sobre cantidades de caracteres a las expresiones regulares

  * → cero o más veces

  + → una o más veces

  ? → cero o una vez

/la+/g

Hola Mundial 12345 → Coincidencia ←→ la

Expresiones Regulares

Patrones Validos:

- Caracteres de escape.- sirven para localizar caracteres no imprimibles y para representar caracteres con significado especial en la expresión.

  \ → carácter de escape

Expresiones Regulares

Patrones Validos:

- Clases o conjuntos de caracteres.- sirven para localizar cualquiera de los caracteres que forman parte del conjunto .

[*conjunto de caracteres*] → Caracteres que pueden formar parte de la cadena

Ej.- [A-Za-z]

conjunto de caracteres\b → Válido si una palabra de la cadena acaba en conjunto de caracteres

/[a-f]\b/g

Hola Mundial 12345 → Coincidencia ←→ a

conjunto de caracteres\B → Válido si una palabra de la cadena no acaba en conjunto de caracteres, pero si formando parte de ella.

/[a-f]\B/g

Hola Mundial 12345 → Coincidencia ←→ d, a

Expresiones Regulares

Patrones Validos:

- Caracteres de marcado de posición.- marcan una posición específica dentro de la cadena.

^ → Comienzo de cadena o línea

/^[A-Z]/g

Hola Mundial 12345 → Coincidencia ←→ H

$ → final de cadena o línea

/[0-9]$/g

Hola Mundial 12345 → Coincidencia ←→ 5

Expresiones Regulares

Patrones Validos:

- Alternancias.- permiten definir opciones para una expresión

/*[a|b]*\b/g → Acaba en *a* ó *b*

Hola Mundial 12345 → Coincidencia ←→ a

Expresiones Regulares

Patrones Validos:

- Agrupaciones.- permiten capturar subpatrones, dentro de un patrón

  (*patrón*)

  Ej.- (A-Za-z)

Expresiones Regulares

Patrones Válidos:

- Subpatrones en cadenas- permiten definir opciones para una expresión

?: → Busca coincidencias con la expression:

/[a-z](?: [A-Z])/g

Hola Mundial 12345 → Coincidencia ←→ a M

Ojo → Espacio en blanco

/[a-z](?: [0-9])/g

Hola Mundial 12345 → Coincidencia ←→ l 1

Expresiones Regulares

Patrones Válidos:

- Subpatrones en cadenas- permiten definir opciones para una expresión

?= → Busca la primera coincidencia de la expression anterior, antes de la siguiente expresión y la captura:

/[a-z](?= [A-Z])/g

Hola Mundial 12345 → Coincidencia ←→ a

Ojo → Espacio en blanco

?! → Busca la primera no coincidencia de la expression anterior, antes de la siguiente expresión y la captura:

/[a-z](?! [A-Z])/g

Hola Mundial 12345 → Coincidencia ←→ o, l, u, n, d, i, a, l

Ojo → Espacio en blanco

Expresiones Regulares

Patrones Validos:

Otros patrones

- {n} → coincide exactamente n veces.
- {n,} → coincide como mínimo n veces.
- {n, m} → coincide n veces mínimo y m veces máximo.

/[a-z] {2,}/g

Hola Mundial 12345 → Coincidencia ←→ ola, undial

Expresiones Regulares

Patrones Validos:

Otros patrones

- /\d/g ←→ /[0-9]/g → Hola Mundial 12345
- /\w/g ←→ /[A-Za-z0-9_]/g → Hola Mundial 12345
- /\W/g ←→ niega /\w/g → Hola_Mundial_12345

Ojo → Espacio en blanco

Expresiones Regulares

Patrones Validos:

Otros patrones

- \n → coincide con carácter nueva línea.
- \r → coincide con retorno de carro.
- \t → coincide con tabulador.
- \f → coincide con avance de página.
- \v → coincide con tabulador vertical.
- \s → coincide con [\n\r\t\f\v]

Expresiones Regulares

Precedencia entre operadores:

- \
- ( ), [ ]
- *, +, {n}, {n,}, {n, m}
- ^, $
- |

Expresiones Regulares

Método test()

Ejemplo: dirección de e-mail válida.

```
Function isEmail(sEMail)
{
    re= /[a-z0-9\-\.]+@[a-z0-9\-\.]+\.[a-z]{2,}/i

    if (re.test(SEMail)
            return true;
    else
            return false;
}
```

Expresiones Regulares

Método exec()

Ejemplo: partes de una dirección de e-mail.

re= /([a-z0-9\-\.]+)@([a-z0-9\-\.]+\.[a-z]{2,})/i

let email= re.exec(SEMail);

Almacena:

email[0].- dirección completa.
email[1].- usuario.
email[2].- dominio.

Nota:

- RegExp.$1= email[1]
- RegExp.$2= email[2]

Expresiones Regulares

Se pueden utilizar métodos del objeto *String* que trabajan de forma implícita con expresiones regulares. Dichos métodos son:

Método	Acción
.match()	Busca coincidencias usando una expresión regular y devuelve un array con los resultados
.matchAll()	Busca coincidencias usando una expresión regular y devuelve un iterador con los resultados
.search()	Devuelve el índice donde encuentra la primera coincidencia
.replace()	Reemplaza texto en una cadena utilizando una expresión regular, devolviendo la cadena modificada
.replaceAll()	Reemplaza texto en una cadena utilizando una expresión regular, devolviendo la cadena modificada con todas las coincidencias aún sin usar /g
.split()	Trocea una cadena en partes utilizando una expresión regular como separador. Devuelve un array con esas partes.

Expresiones Regulares

Método match()

Ejemplo: partes de una dirección de e-mail.

```
const text= “pablo.tahoces@usc.es”
re= (/([a-z0-9\-\.]+)@([a-z0-9\-\.]+\.[a-z]{2,})/i

let email= text.match(re);
```

Almacena:

email[0].- dirección completa.
email[1].- usuario.
email[2].- dominio.

Expresiones Regulares

Método matchAll()

```
<h1>DAW</h1>
<h2>Escuela Técnica Superior de Ingeniería</h2>
<ol></ol>
```

DAW

Escuela Técnica Superior de Ingeniería

1. el
2. ic
3. de
4. ge
5. ie

```
const txt= document.getElementsByTagName("h2")[0].textContent;
it= getTokens(txt);
visualizaTokens(it);
```

```
function getTokens(txt){
  const re=/[a-l](?:[a-l])/g;
  let it= txt.matchAll(re);

  return it;
}
```

```
function visualizaTokens(it){
  let txt= "";
  for (k of it)
      txt+= "<li>" + k + "</li>";
  document.getElementsByTagName("ol")[0].innerHTML= txt;
}
```

Validación mediante JavaScript

```html
<form action="miPagina.html">
  <label for="fname">Introduce nombre usuario:</label>
  <input type="text" placeholder= "Nombre susario">
  <label for="fname">Contraseña:</label>
  <input type="text" placeholder= "Palabra de paso">
  <input type="submit" value="Enviar">
</form>
```

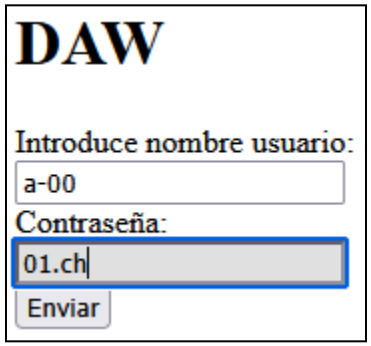
```javascript
const form= document.getElementsByTagName("form")[0];
form.addEventListener("submit", validarForm);
```

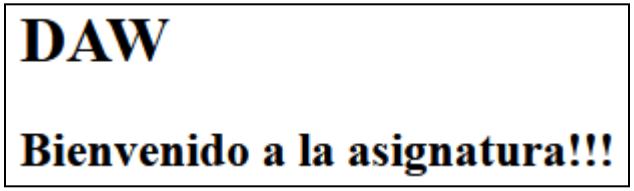
```javascript
function validarForm(e){
    e.preventDefault();
    let msg= "";

    campo= form[0];
    re= /[a-h]{1,3}\-[0-5]{2,3}/i;
    msg= "Introduce un nombre válido";
    if (!re.test(campo.value)){
        changeForm(msg, campo)
        return;
    }

    campo= form[1];
    re= /[0-3]{2,3}\.[ach]{1,2}/i
    msg= "Introduce un pwd válido";
    if (!re.test(campo.value)){
        changeForm(msg, campo)
        return;
    }

    form.submit();
}
```






```javascript
function changeForm(msg, campo){
    alert(msg);
    campo.focus();
    campo.select();
}
```

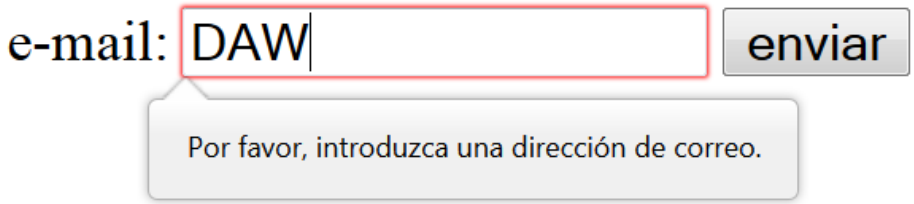
Validación. HTML5.

HTML5 proporciona atributos que permiten realizar validaciones de elementos conocidos para la etiqueta <input>.

Atributo email.- análogo a text, pero para direcciones de email. En caso de introducir una dirección válida, solicita de nuevo un valor.

Ej.-

```
e-mail: <input type="email" name="mail">
<input type="submit" value="enviar">
```



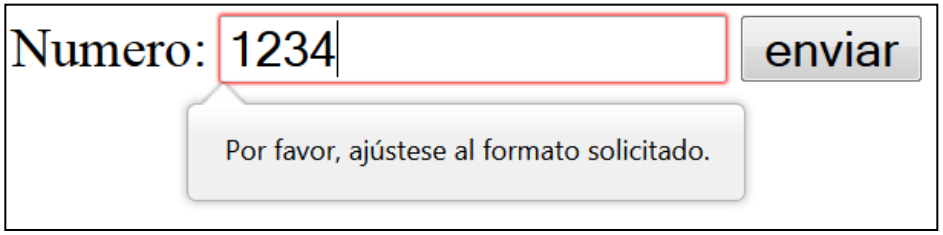
El funcionamiento es análogo para los atributos: url y number

Validación. HTML5.

Atributo pattern.- permite que a la etiqueta <input> le sea insertado un patrón válido en formato de expresión regular.

Ej.-

```
Numero: <input type="number" name="number" pattern="[0-9]{1,3}">
<input type="submit" value="enviar">
```

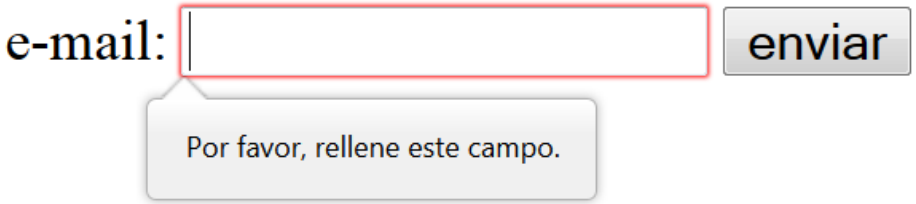


Validación. HTML5.

HTML5 proporciona un atributo para la etiqueta <input> que obliga a rellenar un elemento del formulario → required.

Ej.-

```
e-mail: <input type="email" name="mail“ required>
<input type="submit" value="enviar">
```



Desarrollo de Aplicaciones Web

Expresiones Regulares

USC UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

Pablo García Tahoces